

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 3.

Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), Chapter 6.

3.1 Preliminaries: Searching an Ordered Table

We want to define an interpolation object that knows everything about interpolation except one thing — how to actually interpolate! Then we can plug mathematically different interpolation methods into the object to get different objects sharing a common user interface. A key task common to all objects in this framework is finding your place in the table of x_i 's, given some particular value x at which the function evaluation is desired. It is worth some effort to do this efficiently; otherwise you can easily spend more time searching the table than doing the actual interpolation.

Our highest-level object for one-dimensional interpolation is an abstract base class containing just one function intended to be called by the user: `interp(x)` returns the interpolated function value at x . The base class “promises,” by declaring a virtual function `rawinterp(jlo,x)`, that every derived interpolation class will provide a method for local interpolation when given an appropriate local starting point in the table, an offset `jlo`. Interfacing between `interp` and `rawinterp` must thus be a method for calculating `jlo` from x , that is, for searching the table. In fact, we will use two such methods.

`interp_1d.h`

`struct Base_interp`

Abstract base class used by all interpolation routines in this chapter. Only the routine `interp` is called directly by the user.

```
{
    Int n, mm, jsav, cor, dj;
    const Doub *xx, *yy;
    Base_interp(VecDoub_I &x, const Doub *y, Int m)
    Constructor: Set up for interpolating on a table of x's and y's of length m. Normally called
    by a derived class, not by the user.
        : n(x.size()), mm(m), jsav(0), cor(0), xx(&x[0]), yy(y) {
        dj = MIN(1,(int)pow((Doub)n,0.25));
    }

    Doub interp(Doub x) {
    Given a value x, return an interpolated value, using data pointed to by xx and yy.
        Int jlo = cor ? hunt(x) : locate(x);
        return rawinterp(jlo,x);
    }

    Int locate(const Doub x);           See definitions below.
    Int hunt(const Doub x);

    Doub virtual rawinterp(Int jlo, Doub x) = 0;
    Derived classes provide this as the actual interpolation method.

};
```

Formally, the problem is this: Given an array of abscissas x_j , $j = 0, \dots, N-1$, with the abscissas either monotonically increasing or monotonically decreasing, and given an integer $M \leq N$, and a number x , find an integer j_{lo} such that x is centered

among the M abscissas $x_{j_{lo}}, \dots, x_{j_{lo}+M-1}$. By centered we mean that x lies between x_m and x_{m+1} insofar as possible, where

$$m = j_{lo} + \left\lfloor \frac{M-2}{2} \right\rfloor \quad (3.1.1)$$

By “insofar as possible” we mean that j_{lo} should never be less than zero, nor should $j_{lo} + M - 1$ be greater than $N - 1$.

In most cases, when all is said and done, it is hard to do better than *bisection*, which will find the right place in the table in about $\log_2 N$ tries.

`Int Base_interp::locate(const Doub x)`

`interp_1d.h`

Given a value x , return a value j such that x is (insofar as possible) centered in the subrange $xx[j..j+mm-1]$, where xx is the stored pointer. The values in xx must be monotonic, either increasing or decreasing. The returned value is not less than 0, nor greater than $n-1$.

```
{
    Int ju,jm,jl;
    if (n < 2 || mm < 2 || mm > n) throw("locate size error");
    Bool ascnd=(xx[n-1] >= xx[0]);      True if ascending order of table, false otherwise.
    jl=0;                               Initialize lower
    ju=n-1;                             and upper limits.
    while (ju-jl > 1) {                 If we are not yet done,
        jm = (ju+jl) >> 1;              compute a midpoint,
        if (x >= xx[jm] == ascnd)       and replace either the lower limit
            jl=jm;
        else
            ju=jm;                      or the upper limit, as appropriate.
    }                                    Repeat until the test condition is satisfied.
    cor = abs(jl-jsav) > dj ? 0 : 1;     Decide whether to use hunt or locate next time.
    jsav = jl;
    return MAX(0,MIN(n-mm,jl-((mm-2)>>1)));
}
```

The above `locate` routine accesses the array of values `xx[]` via a pointer stored by the base class. This rather primitive method of access, avoiding the use of a higher-level vector class like `VecDoub`, is here preferable for two reasons: (1) It's usually faster; and (2) for two-dimensional interpolation, we will later need to point directly into a row of a matrix. The peril of this design choice is that it assumes that consecutive values of a vector are stored consecutively, and similarly for consecutive values of a single row of a matrix. See discussion in §1.4.2.

3.1.1 Search with Correlated Values

Experience shows that in many, perhaps even most, applications, interpolation routines are called with nearly identical abscissas on consecutive searches. For example, you may be generating a function that is used on the right-hand side of a differential equation: Most differential equation integrators, as we shall see in Chapter 17, call for right-hand side evaluations at points that hop back and forth a bit, but whose trend moves slowly in the direction of the integration.

In such cases it is wasteful to do a full bisection, *ab initio*, on each call. Much more desirable is to give our base class a tiny bit of intelligence: If it sees two calls that are “close,” it anticipates that the next call will also be. Of course, there must not be too big a penalty if it anticipates wrongly.

The hunt method starts with a guessed position in the table. It first “hunts,” either up or down, in increments of 1, then 2, then 4, etc., until the desired value is bracketed. It then bisection in the bracketed interval. At worst, this routine is about a

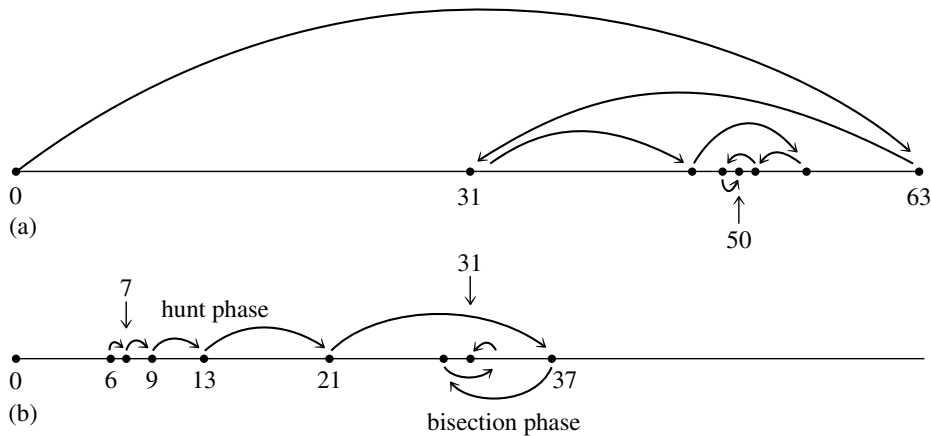


Figure 3.1.1. Finding a table entry by bisection. Shown here is the sequence of steps that converge to element 50 in a table of length 64. (b) The routine `hunt` searches from a previous known position in the table by increasing steps and then converges by bisection. Shown here is a particularly unfavorable example, converging to element 31 from element 6. A favorable example would be convergence to an element near 6, such as 8, which would require just three “hops.”

factor of 2 slower than `locate` above (if the hunt phase expands to include the whole table). At best, it can be a factor of $\log_2 n$ faster than `locate`, if the desired point is usually quite close to the input guess. Figure 3.1.1 compares the two routines.

`interp_1d.h`

`Int Base_interp::hunt(const Doub x)`

Given a value x , return a value j such that x is (insofar as possible) centered in the subrange $xx[j..j+mm-1]$, where xx is the stored pointer. The values in xx must be monotonic, either increasing or decreasing. The returned value is not less than 0, nor greater than $n-1$.

```
{
    Int jl=jsav, jm, ju, inc=1;
    if (n < 2 || mm < 2 || mm > n) throw("hunt size error");
    Bool ascnd=(xx[n-1] >= xx[0]);    True if ascending order of table, false otherwise.
    if (jl < 0 || jl > n-1) {          Input guess not useful. Go immediately to bisection.
        jl=0;
        ju=n-1;
    } else {
        if (x >= xx[jl] == ascnd) {    Hunt up:
            for (;;) {
                ju = jl + inc;
                if (ju >= n-1) { ju = n-1; break;}    Off end of table.
                else if (x < xx[ju] == ascnd) break;    Found bracket.
                else {                                Not done, so double the increment and try again.
                    jl = ju;
                    inc += inc;
                }
            }
        } else {                            Hunt down:
            ju = jl;
            for (;;) {
                jl = jl - inc;
                if (jl <= 0) { jl = 0; break;}    Off end of table.
                else if (x >= xx[jl] == ascnd) break;    Found bracket.
                else {                                Not done, so double the increment and try again.
                    ju = jl;
                    inc += inc;
                }
            }
        }
    }
}
```

```

        }
    }
}
while (ju-jl > 1) {
    jm = (ju+jl) >> 1;
    if (x >= xx[jm] == ascnd)
        jl=jm;
    else
        ju=jm;
}
cor = abs(jl-jsav) > dj ? 0 : 1;
jsav = jl;
return MAX(0,MIN(n-mm,jl-((mm-2)>>1)));
}

```

Hunt is done, so begin the final bisection phase:

Decide whether to use hunt or locate next time.

The methods `locate` and `hunt` each update the boolean variable `cor` in the base class, indicating whether consecutive calls seem correlated. That variable is then used by `interp` to decide whether to use `locate` or `hunt` on the next call. This is all invisible to the user, of course.

3.1.2 Example: Linear Interpolation

You may think that, at this point, we have wandered far from the subject of interpolation methods. To show that we are actually on track, here is a class that efficiently implements piecewise linear interpolation.

`struct Linear_interp : Base_interp`

`interp_linear.h`

Piecewise linear interpolation object. Construct with **x** and **y** vectors, then call `interp` for interpolated values.

```

{
    Linear_interp(VecDoub_I &xxv, VecDoub_I &yv)
        : Base_interp(xv,&yv[0],2) {}
    Doub rawinterp(Int j, Doub x) {
        if (xx[j]==xx[j+1]) return yy[j];
        else return yy[j] + ((x-xx[j])/(xx[j+1]-xx[j]))*(yy[j+1]-yy[j]);
    }
};

```

Table is defective, but we can recover.

You construct a linear interpolation object by declaring an instance with your filled vectors of abscissas x_i and function values $y_i = f(x_i)$,

```

Int n=...;
VecDoub xx(n), yy(n);
...
Linear_interp myfunc(xx,yy);

```

Behind the scenes, the base class constructor is called with $M = 2$ because linear interpolation uses just the two points bracketing a value. Also, pointers to the data are saved. (You must ensure that the vectors `xx` and `yy` don't go out of scope while `myfunc` is in use.)

When you want an interpolated value, it's as simple as

```

Doub x,y;
...
y = myfunc.interp(x);

```

If you have several functions that you want to interpolate, you declare a separate instance of `Linear_interp` for each one.